Sport Assistance

Build a personalized AI sports coach in Python + Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

What you are building **3**

Setup 3

The big picture 4

Build the features **5**

Feature 1: Create an athlete profile 5

Feature 2: Build the personalized coaching prompt 5

Feature 3: Connect the profile to the chatbot 5

Optional extensions **7**

What you are building

Sport Assistance is a chat app with an AI coach that coaches **you**, not "an athlete in general". You fill in a profile (sport, experience, dominant hand, strengths, weaknesses, preferences). The coach reads it before every answer, so a left-handed beginner gets different advice from a right-handed advanced player.

The folder you were given is a **scaffold**: the app already runs in demo mode, and three features are left for you. You will:

1. Turn the form into an **athlete profile** (a Python dictionary).
2. Turn that profile into a **personalized prompt** for the AI.
3. **Connect** the profile and prompt to the chatbot, including the conversation history.

You will not hand-write the interface, the AI call or the sport list. Those are provided, and the next sections show where they fit.

Setup

Open a terminal inside the `sport_assistance` folder (Mac: type `cd ` and drag the folder in; Windows: click the folder's address bar, type `powershell`, press Enter). Then run the commands for your computer one at a time.

Mac:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python app.py
```

Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python app.py
```

Open the address it prints (usually `http://127.0.0.1:7860`). **Every time you edit a file, stop the app with `Ctrl + C` and run `python app.py` again.** Reopen the terminal later? Run the `activate` line again first.

> **STUCK?**
> Windows: if `python` is not found, use `py -m venv .venv`. If PowerShell blocks `activate`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (answer Y) or use Command Prompt.

API key (optional): without a key the app runs in **demo mode**, and the chat shows what *would* be sent to the AI. That is enough to finish every feature. For real AI replies, create a Gemini key at `aistudio.google.com/apikey` (or ask your teacher) and put it in `.env` as `GEMINI_API_KEY=your-key`, with no quotes or spaces. Never put the key in a Python file or share your `.env`.

The big picture

The AI knows **nothing** about you, so our code writes everything it needs into a block of text called a **prompt**:

```
form inputs --> profile dict --> prompt text --> AI model --> reply
(app.py)        (profile.py)     (coach.py)     (ai_helper.py)  (chat window)
```

Gradio keeps the profile and the chat history in a `gr.State`, so they are remembered while the page is open. The AI has no memory either, so the history is sent again with every message.

File	Job
`app.py`	The screen: tabs, buttons, and the functions that run when you click. Finished.
`profile.py`	Creates the profile dictionary. **Feature 1 is here.**
`coach.py`	Builds the coaching prompt and the reply. **Features 2 and 3 are here.**
`ai_helper.py`	The only file that talks to the AI. You never edit it.

Tabs 2 (Playing style) and 4 (Update profile) already work, so ignore them for now. Open `app.py` and read `handle_save_profile`: it shows how a button click becomes a Python function call and then new text on screen.

Build the features

Feature 1: Create an athlete profile

File: `profile.py` , function `create_profile`

Goal: turn the six form values into a profile dictionary that the rest of the app can use.

Your Task: finish the two `TODO` s. Store the handedness only for sports where it matters (for dance it stays `"N/A"`). Store strengths and weaknesses as **lists**, not as one long string, and store preferences without extra spaces around them.

Hint: `side_matters(sport)` gives `True` or `False` , which is exactly what an `if` needs. `split_list(text)` is a helper that already turns comma text into a list.

Test: restart, open tab 1, choose **tennis**, **Left**, strengths `serve, footwork` , and click **Save profile**. The summary should show Left. Then check the data itself:

```
python -c "from profile import create_profile; print(create_profile('tennis', 'Beginner', 'Left',
    'serve, footwork', 'backhand', ' short drills '))"
```

You should see `'handedness': 'Left'` , `['serve', 'footwork']` and `'short drills'` . Try `'dance'` instead: handedness must be `'N/A'` .

Feature 2: Build the personalized coaching prompt

File: `coach.py` , function `build_coaching_system_prompt`

Goal: return the text that tells the AI who it is and who it is coaching. This is called the **system prompt**, and it is where the personalization happens.

Your Task: the starter already says "You are a ... coach" and lists the profile. Finish the two `TODO` s. First, add extra instructions only when they apply: a left-handed athlete, a beginner, and any preferences the athlete wrote. Second, add rules for everyone: the coach must say so when something is unsafe or ineffective and suggest a safer option, and answers should stay short and practical.

Hint: build the prompt in pieces with `prompt += "..."` , and put each profile-dependent piece inside an `if` so a right-handed advanced athlete does not get it. `profile.get("handedness")` reads a value from the dictionary.

Test: print the prompt for two different athletes and compare them:

```
python -c "from coach import build_coaching_system_prompt as b; from profile import
    create_profile as c; print(b(c('tennis', 'Beginner', 'Left', 'serve', 'backhand', 'short
        drills')))"
```

Read it as if you were the AI. It should mention left-handed, beginner, short drills, a safety rule and a brevity rule. Change `'Beginner'` to `'Advanced'` and `'Left'` to `'Right'` : the left-handed and beginner lines must disappear.

Feature 3: Connect the profile to the chatbot

File: `coach.py` , function `get_coaching_response`

Goal: make the chat answer using the athlete's profile **and** the earlier conversation, so a follow-up like "What about my left hand?" makes sense.

Your Task: finish the three `TODO`s. Build the system prompt for this athlete, make the full list of messages to send (the earlier history plus the new user message), and send both to the AI function that is already imported at the top of the file. Return what it gives back.

Hint: each message is a dictionary with a `"role"` and a `"content"` key, exactly like the items already in `history`. You can make a new list by adding two lists together.

Test: restart, save a profile in tab 1, go to tab 3 and send a message. In demo mode the reply shows your system prompt and says it would send **1 message(s)**. Send a second message: it should now say **3 message(s)** (your first question, the coach's reply, and your new one). That proves the history is being sent. With a real key, ask *How can I improve my serve?* and then *What about my left hand?*; the second answer should make sense.

Optional extensions

If you finish early, pick any of these. They are open-ended, so design them your own way.

1. **A more careful style analysis.** In tab 2, the AI summary comes from `build_style_analysis_prompt`, which is basic. Rewrite it so the AI does not invent facts, words its guesses as possibilities ("may", "seems"), and asks the athlete to confirm. If `include_pro_comparisons` is on, add a section clearly labelled as *not verified*.
2. **A new sport.** Add one to `SPORTS` in `app.py` and give it coaching topics in `coach.py`. Do you need to change `side_matters`?
3. **Coach personalities.** Add a dropdown (for example "Encouraging" or "Strict but fair") that changes a line in the system prompt.
4. **Goals.** Add a list of goals to the profile and mention them in the prompt. What has to change in `create_profile`, `profile_to_text` and the form?
5. **Streaming replies.** Make the text appear as it is written, using `generate_content_stream` in the Gemini library and `yield` in a Gradio function. This touches `ai_helper.py`.
6. **Try another AI.** The app was built so only `ai_helper.py` knows which AI it talks to. Switch it to a different provider and check that nothing else breaks.